A few properties, such as `cluster.election.duration` and `cluster.election.initial_timeout`, help us configure the frequency of elections and how long to wait before master-eligible nodes call for an election. The `initial_timeout` attribute, for example, is the amount of time a master-eligible node waits before calling for an election. By default, this value is set to 500 ms. For example, let's say master-eligible node A does not receive a heartbeat from the master node in 500 ms. It then calls for an election because it thinks the master has crashed.

In addition to electing a master, master-eligible nodes work together to get cluster operations rolling. Although the master is the king of the cluster, it needs support and buy-in from the master-eligible nodes. The master's job is maintaining and managing cluster state, as we see next.

14.6.3 Cluster state

The cluster state consists of all the metadata about shards, replicas, schemas, mappings, field information, and more. These details are stored as the global state in the cluster and are also written to each node. The master is the only node that can commit the cluster state. It has the responsibility to maintain the cluster with up-to-date information. Master nodes commit the cluster data in phases (similar to the two-phase commit transaction in a distributed architecture):

1. The master computes the cluster changes, publishes them to the individual nodes, and then waits for acknowledgments.
2. Each node receives the cluster updates, but the updates aren't applied to the nodes' local state yet. On receipt, they send an acknowledgment to the master.
3. When the master receives a quorum of acknowledgments from the master-eligible nodes, it commits the changes to update the cluster state (the master doesn't need to wait for acknowledgments from every node, just master-eligible nodes).
4. After successfully committing the cluster changes, the master node broadcasts a final message to individual nodes instructing them to commit the previously received cluster changes.
5. The individual nodes commit the cluster updates.

The `cluster.publish.timeout` attribute sets a time limit (30 s by default) to commit each batch of cluster updates successfully. This period runs from the time of posting the publication of the first cluster update messages to the nodes until the cluster state is committed. If the global cluster updates are committed successfully within the default 30 s, the master waits until this time elapses before starting the next batch of cluster updates. However, the story doesn't end here.

If the cluster updates are not committed in 30 s, the master may have died. The election for a new master begins as a result.

Although the global cluster updates are committed, the master still waits for nodes that haven't returned acknowledgments. Unless it receives acknowledgments, the